PHP-Symfony-Laravel — compiled path

> Compiled from `/Users/darkonikolic/Documents/learning/paths/PHP-Symfony-Laravel`. Canonical sources stay in place.

Phase — 01-modern-php-foundations-professional-shape

Directory: `01-modern-php-foundations-professional-shape/`

Source: `01-modern-php-foundations-professional-shape/01-scope-from-endpoints-to-maintainable-systems.md`

Unit 1 — Scope: from “shipping endpoints” to owning a maintainable system

Goal

Rewire instinctive progress from CRUD familiarity to **system ownership**:

- Bounded behaviour, observable boundaries, reproducible deployments, intentional coupling at integration seams.

Professional PHP is rarely “better syntax”: it is **composition, contracts, predictable failure semantics, dependency governance**.

Deliverable

Draft a one-page charter for **one hypothetical service**:

- Ownership boundaries versus shared libraries
- Explicit error taxonomy (recoverable vs operator vs bug)
- What you refuse to automate without tests

Interview stance

Articulate trade-offs behind “thin controllers” versus “thick domain”: when each becomes honest versus cargo-cult scaffolding.

Source: `01-modern-php-foundations-professional-shape/02-modern-language-constructs-impact-on-design-enums-and-attributes.md`

Unit 2 — Language surface that shapes backend design

Focus

Treat language features as **design levers**:

- `enum` **backed cases for state**, not sprawling string constants.
- `Attribute` **s** documenting cross-cutting semantics (routing metadata, serialization policy, doctrine mapping discipline) versus anonymous magic conventions.
- `Generator` **iterators**: streaming ingestion with bounded memory; still model **lifetime and exception semantics** (consumer abort mid-sequence vs full materialisation pitfalls).
- **Exceptions as rare control flow**: domain failures often deserve **explicit result types**, not swallowed stack traces everywhere.

Lab

Specify **three** refactor targets in legacy-style code where:

- enums replace sentinel strings safely,
- an attribute declares one cross-cutting invariant you currently encode in procedural checks.

Deliverable short note on **backward compatibility drift** risks when altering enum cases or widening attribute interpretations.

Source: `01-modern-php-foundations-professional-shape/03-oop-composition-interfaces-traits-contract-discipline.md`

Unit 3 — OOP: interfaces, traits, composition without framework worship

Outcomes

- **Interfaces survive longer than adapters**: carve boundaries where mocking is honest, not “interface per class ritual”.
- **Traits as mechanical reuse only**: visible behaviour still needs tests; forbid god-trait hierarchies collapsing boundaries.
- **Composition over ornamental inheritance**.
- Understand **Variance** intuitively (`iterable<T>` ergonomics affecting API typing).

Drill

Enumerate **five** codebase smells where Traits hide dependencies you should expose through constructor contracts.

Interview: Explain when **explicit final classes + narrow interfaces** slow teams down positively versus needless ceremony.

Source: `01-modern-php-foundations-professional-shape/04-composer-autoload-dependencies-and-graph-governance.md`

Unit 4 — Composer: reproducible graphs, autonomy, governance

Outcomes

- `composer.json` **declares intent**, `composer.lock` pins reality—articulate rollback strategy when merges conflict on lock divergence.
- **Autoload semantics** (`psr-4` mapping discipline) aligning namespace roots with folder reality—prevent ghost directories.
- **Platform config** pinning PHP extensions—avoid surprises in CI parity.
- **Replace / path repos** sparingly document escape hatches risking supply-chain drift—when local override is ethically justified temporarily.

Lab

Conduct an **upgrade rehearsal** sandbox:

```
composer outdated --direct
```

Pick one non-major bump. Document:

- breaking risk scan (changelog skim),
- test matrix delta,
- whether you’d batch or isolate release.

Interview: Discuss **supply chain vigilance**: integrity constraints, verifying sources, narrowing dependency fan-out intentionally.

Source: `01-modern-php-foundations-professional-shape/05-lab-explicit-errors-mini-domain-model.md`

Unit 5 — Lab: miniature domain slice with explicit invariants

Brief

Implement (or blueprint in pseudocode structured enough that you could paste into a spike repo) **one bounded slice**: e.g. `OrderReservation` issuing stock holds with deterministic failure modes.

Requirements

- Entities / value objects (language-level classes) encapsulate invariants—not generic arrays everywhere.
- **No framework ORM leakage** inside the pure calculation core (Doctrine / Eloquent only at edges later).
- **Explicit failure mapping** surfaced to hypothetical HTTP layer—not generic `RuntimeException`.

Acceptance checklist

Domain core unit tests asserting:

- invariant violation paths,
- deterministic output for happy path transitions,
- no hidden globals / static façade dependencies.

Phase — 02-symfony-expert-backend-core

Directory: `02-symfony-expert-backend-core/`

Source: `02-symfony-expert-backend-core/01-scope-kernel-request-lifecycle-professional-reading.md`

Unit 1 — Symfony runtime: Kernel, Requests, Responses as contracts

Senior outcomes

- Navigate **compiled container** confidently: understand config vs semantic config vs env-specific layering without treating cache warming as folklore.
- **Request / Response semantics** bridging HTTP nuances (trusted headers proxies, stale session edge cases)—security implications not hand-waved.

Drill

Produce a textual sequence diagram traversing kernel events you'd instrument **first** diagnosing:

- unexplained duplicated controller execution suspicion,
- early termination path missing expected listener.

Interview: Explain **Symfony HttpKernel portability** rationale vs micro-framework zeal—trade-off clarity earns senior signal.

Source: `02-symfony-expert-backend-core/02-di-container-tags-decorators-and-explicit-wiring.md`

Unit 2 — Dependency injection mastery beyond autowire defaults

Senior depth

- **Autowiring heuristics** limitations: collisions, ambiguous service ids, generics typing blind spots requiring explicit binds.
- **Decorators**: layered cross-cutting collaborators (audit, metering) without rewriting core handlers.
- `#[Autoconfigure]` / `#[Exclude]` **nuances** sparingly—not attribute soup replacing architectural thinking.
- **Compiler passes & tagged iterator patterns** consolidating plugin-like extension strategies.

Practice

Pick one cross-cutting collaborator (audit / metrics façade). Outline decorator layering **before** proposing event listener proliferation—justify compositional coherence.

Source: `02-symfony-expert-backend-core/03-messenger-async-transports-retries-serializers-and-consistency-notes.md`

Unit 3 — Messenger professional transport design

Goals

- **Handler idempotency** across redelivery horizons—timeouts vs logical duplicates.
- **Serialization boundaries**: class-based vs DTO payloads; upgrade compatibility when renaming message classes responsibly.
- **Failure queues / DLQ thinking**: distinguishing poison messages vs infra blips versus partial commit hazards.
- **Transactional integration**: aligning DB commit horizons with enqueue visibility (bridge to transactional outbox later).

Symfony-specific angle

Know when **sync vs async** transports change failure semantics subtly (kernel vs worker process boundaries).

Source: `02-symfony-expert-backend-core/04-domain-and-application-hook-points-events-vs-messenger.md`

Unit 4 — Events versus commands: coherence not buzzwords

Separate design lanes

Concern kind	Typical shape
Domain reaction synchronous side-effect orchestration guarded by aggregate rules	Controlled inside application transaction
Integration fan-out	Message / event after consistent state persists
Observability augmentation	Lightweight listeners sparing global mutation

Symfony EventDispatcher interplay

- Understand **stopped propagation semantics** pitfalls.
- Choose **callable listener wiring** ergonomics balancing discoverability versus explicit service definitions.

Source: `02-symfony-expert-backend-core/05-doctrine-orm-professional-mapping-transactions-and-fetch-discipline.md`

Unit 5 — Doctrine at senior depth (not accidental Active Record)

Understand

- **Unit of Work** mental model identity map staleness pitfalls when mixing raw SQL casually.
- **Lazy vs eager** loading trade-offs & N+1 recognition patterns—not only profiler blame.
- **Transaction boundaries spanning domain + integrations**: flushing order vs messenger dispatch bridging (consistency pitfalls).
- **Mapping inheritance strategies** repercussions (JOINED vs SINGLE_TABLE) altering query shape subtly.

Interview talking points

Hydration explosions, partial object graphs via DTO projections, **read model vs aggregate write model** divergence.

Source: `02-symfony-expert-backend-core/06-security-firewalls-authenticators-voters-and-access-rules.md`

Unit 6 — Security component: hardened mental map

Expert expectations

- **Firewall rules entry points** aligning with layered reverse proxy trust boundaries.
- **Authenticator pipeline** customizing failure differentiation (credential vs escalation vs outage) without dumping stack traces externally.
- **Voters granular authorization** aligning policy statements with aggregates / commands—not controller-only `ROLE_*`.
- `#[IsGranted]` **& expression language** sparingly balancing declarative ergonomics versus hidden branching complexity.

Operational angle

Discuss **logout / session fixation hardening nuances**, remember-me pitfalls, brute-force signalling integration readiness.

Source: `02-symfony-expert-backend-core/07-framework-cache-surfaces-consistency-awareness.md`

Unit 7 — Cache layers Doctrine & HTTP coherence

Goals

- Distinguish **application cache adapters** versus **Doctrine result / query caching** semantics (staleness horizons differ).
- **HTTP cache layering** interplay (ETag negotiation, surrogate keys conceptually)—even API-only backends may emit conditional semantics thoughtfully.

- **Invalidation choreography** aligning domain events versus brute TTL-only strategies.

Interview

Enumerate **failure classes** introducing ghost reads after deployments if cache layers misaligned schema evolution.

Source: `02-symfony-expert-backend-core/08-lab-vertical-slice-payment-intent-gateway-symfony.md`

Unit 8 — Lab: ecommerce payment intent initiation (Symfony)

Scenario

Expose **payment intent initiation** enforcing:

- transactional guard around draft order state transition,
- async notification via Messenger after commit,
- idempotent handler handling duplicate enqueue.

Deliverables

Diagram + folder outline showing **explicit layers** separating domain reasoning from infra glue.

Interview reflection

Enumerate **five** regressions testers should script if event ordering breaks subtly.

Phase — 03-laravel-expert-backend-core

Directory: `03-laravel-expert-backend-core/`

Source: `03-laravel-expert-backend-core/01-scope-framework-lifecycle-and-http.md`

Unit 1 — Laravel lifecycle mastery

Senior expectations

- **Service provider layering** respecting boot vs register ordering side-effects—prevent hidden global mutation during tests.
- **Request lifecycle + middleware pipelines** aligning cross-cutting sequencing (termination callbacks awareness).
- **Configuration caching** interplay in production—not toggling blindly in CI vs local parity gaps.

Interview

Contrast **Symfony compiled container sophistication** versus **Laravel's pragmatic resolution** ergonomics articulating pragmatic trade-offs without tribal flame wars.

Source: `03-laravel-expert-backend-core/02-service-providers-and-container-extension-points.md`

Unit 2 — Providers binding contracts thoughtfully

Goals

- **Deferred vs eager binding** ramifications for cold start latency.
- `singleton` **leakage vs scoped instances** aligning long-lived connectors with test isolation.
- **Contextual bindings** sparingly bridging multi-tenant or multi-queue drivers cleanly.

Practice note

Enumerate **four** Laravel conveniences enticing hidden static singletons harmful to deterministic tests—remediation strategy.

Source: `03-laravel-expert-backend-core/03-queue-jobs-batch-and-chain-reliability.md`

Unit 3 — Queued jobs as distributed transactions' distant cousins

Depth

- **Job middleware** structuring retry / timeout policy consistency—avoid scattering `public $timeout` arbitrarily.
- **Batch semantics** bridging partial successes / compensations—when saga-like choreography emerges from naive fan-out failures.
- **Failed job inspection discipline** aligning ops expectations with code-level naming & payload clarity.
- `ShouldQueue` **listeners** interplay with transactional DB boundaries identical conceptual hazards as Symfony bridging.

Source: `03-laravel-expert-backend-core/04-domain-events-vs-application-events-queue-edges.md`

Unit 4 — Events subsystem: cohesion under load

Articulate distinctions

Domain events emitted inside aggregate commits versus **broadcast-style integration chatter**. Laravel's eloquent observers tempt silent domain leakage—articulate refactor path toward intentional application emitters saturating transactional boundaries cleanly.

Source: `03-laravel-expert-backend-core/05-eloquent-query-scopes-aggregates-and-repository-discipline.md`

Unit 5 — Eloquent disciplined at bounded edges

Expert themes

- **Repository façade** guarding query explosion entry points—not dogmatic forbidding Models but resisting unbounded chaining from controllers arbitrarily.
- **Scopes & builder composition** aligning maintainable SQL ergonomics readable by reviewer—avoid mystical dynamic global scopes collapsing traceability unintentionally.
- **Aggregate cohesion vs anemic duplication**: choose transactional boundaries thoughtfully when multiple models interplay.
- **Database-level constraints** aligning with eloquent validations—dual validation layers coherence.

Interview

Describe **silent partial update pitfalls** circumventing eloquent timestamps & observer hooks using raw persistence alternatives.

Source: `03-laravel-expert-backend-core/06-policies-gates-multi-guard-and-authorization-shape.md`

Unit 6 — Authorization: policies gates multi-guards

Goals

Map **Symfony Voter mental model → Laravel Policies / Gates**:

- aligning subject + user context ergonomics cleanly,
- **after middleware vs route-model implicit binding interplay** guarding attribute-level checks,
- **resource collections** iterating authorization without N policy roundtrips careless patterns.

Operational angle

API token abilities vs SPA session distinctions—articulate escalation boundaries.

Source: `03-laravel-expert-backend-core/07-lab-vertical-slice-payment-intent-gateway-laravel.md`

Unit 7 — Lab: same payment intent initiation (Laravel twin)

Rebuild **Symfony lab slice** equivalents:

- guarded state transition initiating payment intent persisted transactionally,
- queued async notification respecting commit ordering,
- idempotent retry path.

Difference inventory

Enumerate **seven** divergence points Laravel introduces vs Symfony ergonomically affecting test harness shape & failure attribution clarity.

Phase — 04-enterprise-architecture-php-ddd-and-cqrs

Directory: `04-enterprise-architecture-php-ddd-and-cqrs/`

Source: `04-enterprise-architecture-php-ddd-and-cqrs/01-ddd-strategic-context-map-and-integration-patterns.md`

Unit 1 — Strategic design in PHP backends

Produce

- Bounded context catalogue draft for hypothetical marketplace (catalogue, fulfilment, billing).
- Identify **upstream / downstream coupling** directional commitments (customer–supplier versus conformist versus ACL).

Deliverable textual **context map** plus three explicit anti-patterns you'd outlaw on day one culturally.

Source: `04-enterprise-architecture-php-ddd-and-cqrs/02-tactical-models-and-validation-at-integration-edges.md`

Unit 2 — Tactical building blocks at framework edges

Focus

- Entities encapsulating invariants; value objects for money / quantities / identifiers when semantics matter.
- Application services orchestrating **use cases** without bloating controllers or models.
- **Validation orchestration layering**: inbound DTO assertions vs invariant enforcement differences.

Discuss **Symfony Form / Serializer** vs **Laravel Requests + Resources / API Resources mapping** symmetrical cautions bridging domain internals exposure.

Source: `04-enterprise-architecture-php-ddd-and-cqrs/03-cqrs-command-query-separation-bus-and-handlers.md`

Unit 3 — CQRS separation without ceremonial dogma

Outcomes

- Command handling single write intent with explicit transaction boundary acknowledgement.
- Read models optimized for UI / API ergonomics—even if Doctrine / Eloquent still backs them pragmatically initially.
- **Bus indirection rationale**: delaying framework-specific handler discovery coupling.

Discuss **Symfony Messenger commands** parallels **Laravel command buses** optionally + event sourcing caution footnote—not mandatory stack leap.

Source: `04-enterprise-architecture-php-ddd-and-cqrs/04-hexagonal-adapters-with-symfony-and-laravel-drivers.md`

Unit 4 — Hexagonal architecture framework adapters

Map **ports**:

- Persistence port implementation swap (Doctrine Repository vs Eloquent repository façade).
- External payment gateway façade isolating timeouts / idempotency key generation.
- Email / SMS notification port mocking contract tests cleanly.

Articulate dependency direction rule **domain core never referencing HTTP attributes**.

Source: `04-enterprise-architecture-php-ddd-and-cqrs/05-repository-specifications-and-regeneration-factory-logic.md`

Unit 5 — Repository Specification Factory trade-offs

Discuss

- Specifications composing query predicates—when they collapse readability versus clarifying duplication.
- Factory patterns rebuilding aggregates from reconstructed events / rows—risk of divergence if partial hydration inconsistent.
- **Anti-corruption layering** guarding legacy schema shapes.

Source: `04-enterprise-architecture-php-ddd-and-cqrs/06-domain-vs-integration-domain-events-survival-rules.md`

Unit 6 — Domain versus integration domain events taxonomy

Articulate envelopes

Persistent **outbox-backed events** guaranteeing at-least-once consumer semantics versus ephemeral kernel dispatch broadcast noise.

Enumerate **replay hazards** renaming event class contracts without compatibility strategy.

Source:
`04-enterprise-architecture-php-ddd-and-cqrs/07-lab-ecommerce-modular-boundaries-and-integration-flow.md`

Unit 7 — Lab: ecommerce module vertical slice narration

Produce architecture note (no mandatory full codebase) describing ecommerce vertical slice decomposition:

Cart → Checkout initiation → Inventory reservation saga outline → Billing capture stub.

Identify **Symfony vs Laravel specific adapter insertion points**.

Acceptance checklist

- Saga compensation narrative for partial failure—not only happy choreography.

Phase — 05-api-contract-engineering-rest-and-openapi

Directory: `05-api-contract-engineering-rest-and-openapi/`

Source: `05-api-contract-engineering-rest-and-openapi/01-http-resource-shape-pagination-filtering-professional-shape.md`

Unit 1 — REST surface discipline professional expectations

Articulate coherent resource narratives beyond CRUD synonyms.

Depth topics

Concern	Professional nuance
Filtering	Stable query contract vs ad hoc exploding parameter sets
Sorting	Indexed column alignment awareness
Pagination	Offset pitfalls vs opaque cursor ergonomics exposing leak risks mentally
Error bodies	Stable machine-readable problem structure vs leaking stack internals

Enumerate **seven** versioning-adjacent header strategies (Deprecation, Sunset)—even if delaying adoption.

Source: `05-api-contract-engineering-rest-and-openapi/02-openapi-contract-consumer-guardrails.md`

Unit 2 — OpenAPI as contract truth distillation

Practice angle

Define schema diff policy on PR:

- Breaking vs additive classifications,
- Automated consumer pact alignment hook conceptually—even if phased adoption.

Discuss **Symfony API Platform ergonomics intersections** optionally versus **manual attribute mapping** disciplined minimalism Laravel route binding analogues—not vendor training.

Source: `05-api-contract-engineering-rest-and-openapi/03-idempotency-and-safe-retries-professional-shape.md`

Unit 3 — Idempotency keys and commanding semantics

Outcomes

- HTTP idempotency token headers reconciling duplicated client retries aligning with CQRS handlers.
- Distinguishing **natural idempotency** (PUT replace) versus **constructed idempotency** (financial commands).
- Persisted idempotency record retention / GC horizons governance.

Source: `05-api-contract-engineering-rest-and-openapi/04-lab-public-api-stable-under-change.md`

Unit 4 — Lab: hardened public-ish API façade

Produce contract outline for hypothetical **merchant inventory adjustment** mutation:

Acceptance artefacts

OpenAPI excerpt + three negative test narratives (quota exceeded, precondition failed, concurrency conflict surfaced intentionally).

Phase — 06-async-messaging-integration-and-workers

Directory: `06-async-messaging-integration-and-workers/`

Source: `06-async-messaging-integration-and-workers/01-transactional-outbox-double-commit-hazards.md`

Unit 1 — Transactional honesty before hero workers

Articulate saga failure classes when DB commits succeed but message publication fails—or vice versa.

Implementation directions

Symfony **Doctrine transactional middleware** interplay vs Laravel transactional closure + dispatched events ordering discipline.

Source: `06-async-messaging-integration-and-workers/02-retry-dlq-semantics-professional-reading.md`

Unit 2 — Retries backoff jitter poison messages

Enumerate strategies

Linear vs exponential backoff, jitter rationale, capped attempts, alerting on DLQ swell rates.

Operational clarity

Correlation identifiers traversing synchronous + asynchronous hops for cross-surface RCA narrative.

Source: `06-async-messaging-integration-and-workers/03-lab-email-analytics-and-payment-async-wiring-narratives.md`

Unit 3 — Lab trilogy: integrations fan-out realistically

Stories (written architecture + sequence rationale — code optional spike)

Analytics event emission after fulfilled order

Email activation after registration with retry storm guard

Payment capture confirmation reconciliation worker idempotency storyline

---

Phase — 07-performance-profiling-dataplane-and-php-runtime

Directory: `07-performance-profiling-dataplane-and-php-runtime/`

Source: `07-performance-profiling-dataplane-and-php-runtime/01-doctrine-query-profiler-and-batch-discipline.md`

Unit 1 — Doctrine / ORM query-plane awareness (foundations for performance)

Professional reading

- Interpreter query plans alignment with ORM expectations—not surprising the database at scale.
- Hydration cost when mapping wide rows into rich graphs—DTO / partial projection trade-offs.
- Batch operations & flushing strategies reducing round trips without silent integrity loss.
- Observability hooking: slow query logging thresholds, integration with APM / profiler trace correlation.

For Laravel practitioners: map **N+1 symptom resolution** patterns analogously even if tooling names differ.

Source: `07-performance-profiling-dataplane-and-php-runtime/02-framework-cache-http-and-opcache.md`

Unit 2 — Caching strata + PHP runtime hotspots

Articulate distinctions

Symfony HttpCache surrogate awareness vs Laravel response cache middleware analogues pragmatically—not identical machinery.

Opcache preload / JIT high-level intuition

When enabling aggressive opts risks deployment coupling surprise.

Source: `07-performance-profiling-dataplane-and-php-runtime/03-lab-remediate-slow-hot-read-endpoint-evidence-backed.md`

Unit 3 — Lab: slow API triage artefacts

Assume endpoint latency regression.

Produce investigation timeline stub:

Profiler capture → SQL diff → caching hypothesis → invalidated wrong layer discovery narrative.

Phase — 08-testing-pyramid-contracts-integration-and-browser-truth

Directory: `08-testing-pyramid-contracts-integration-and-browser-truth/`

Source: `08-testing-pyramid-contracts-integration-and-browser-truth/01-pyramid-shape-symfony-laravel-aligned.md`

Unit 1 — Testing pyramid fidelity for frameworks

Articulate pragmatic boundaries

Symfony **KernelTestCase** layering vs Laravel **PHPUnit + HTTP tests**.

When pure unit purity yields false confidence missing container wiring regressions responsibly.

Source:
`08-testing-pyramid-contracts-integration-and-browser-truth/02-contract-consumer-producer-safety-php.md`

Unit 2 — Contract Testing integration surfaces

Discuss schema / message contract evolution safety nets protecting multi-team deployment cadence—even monolith internal module boundaries benefiting analogously sometimes.

Source: `08-testing-pyramid-contracts-integration-and-browser-truth/03-functional-e2e-panther-dusk-tradeoffs-and-flakes.md`

Unit 3 — Higher layers without flake hell discipline

Enumerate flake sources

timing assumptions, asynchronous UI partial readiness (if SPA adjacent), brittle DOM coupling.

Articulate shrinking strategy isolating deterministic API contract tests reserving browser truth minimally.

Source: `08-testing-pyramid-contracts-integration-and-browser-truth/04-lab-tdd-narrative-for-critical-module-scope.md`

Unit 4 — Lab TDD narration

Pick hypothetical pricing rule module outlining failing tests first ordering:

pure domain rules → transactional integration boundary → façade HTTP contract minimally.

Phase — 09-backend-security-threat-model-and-hard-auth

Directory: `09-backend-security-threat-model-and-hard-auth/`

Source: `09-backend-security-threat-model-and-hard-auth/01-threat-model-apis-and-dependencies.md`

Unit 1 — Threat modelling backend surfaces

Enumerate STRIDE-relevant bullets adapted pragmatically—not academic theatre.

Discuss dependency escalation attack surface widening via careless composer merges.

Source: `09-backend-security-threat-model-and-hard-auth/02-oauth-openid-session-and-jwt-hardened-tradeoffs.md`

Unit 2 — Authentication stacks compared professionally

Articulate pitfalls

JWT misuse forgetting rotation / replay resistance, SPA token storage traps, SSRF gateways during OAuth dances.

Symmetric mapping of Symfony OAuth integrations vs Laravel Sanctum / Passport conceptual territory without prescriptive vendor lock narration.

Source: `09-backend-security-threat-model-and-hard-auth/03-rbac-rate-limiting-abuse-surfaces-secret-hygiene.md`

Unit 3 — Operational security controls

Articulate aligning rate limits with abusive vs legitimate burst patterns.

Discuss secret rotation rehearsals & environment divergence hazards.

Phase — 10-production-rollouts-and-interview-synthesis

Directory: `10-production-rollouts-and-interview-synthesis/`

Source: `10-production-rollouts-and-interview-synthesis/01-deploy-migrations-rollbacks-and-parity-thinking.md`

Unit 1 — Deployments migrations rollbacks environment parity

Articulate

Blueprint / forward-only migration mentality vs destructive rollback scripts cautions.

Feature flag interplay with schema evolution coupling risk.

Zero-downtime multi-phase column introduction narrative (expand → backfill → contract → contract switch → remove old).

Symfony **doctrine:migrations** ergonomics vs Laravel migration ordering discipline—fail-forward communication culture.

Source: `10-production-rollouts-and-interview-synthesis/02-ci-artefacts-static-analysis-and-shift-left.md`

Unit 2 — CI/CD quality gates pragmatic professional shape

Articulate layering

composer audit hooks, phpstan/psalm deepening thresholds responsibly, ephemeral integration DB spin-up costs trade-offs.

Source: `10-production-rollouts-and-interview-synthesis/03-interview-incident-thinking-deadlocks-and-scaling-prompts.md`

Unit 3 — Interview-ready synthesis bridging theory + PHP stacks

Articulate crisply

SOLID applicability limits in evolving systems

DDD heuristic recall without pattern theatre

Doctrine deadlock narratives + queue retry amplification interplay

Scaling paths: horizon workers, concurrency cautions caching stampede bridging.

Produce **six** probing self-questions unanswered yet—your honest backlog sharpening focus.
